

Aufgabe 1: Hopsitexte

Team-ID: 00233

Team: Bougainvillea

Bearbeiter/-innen dieser Aufgabe:

Tao Zheng

2. November 2024

Inhaltsverzeichnis

1 Lösungsidee	1
2 Umsetzung	2
2.1 Funktionen, um die Buchstaben zu konvertieren	2
2.2 Anzeigepositionen	2
2.3 Weitere Funktionen	3
2.4 Stapelspeicher	3
2.5 Eingabe & Schleife	3
3 Beispiele	4
3.1 Beispiele aus der PDF-Datei	4
3.2 Beispiel mit ungültigen Zeichen	5
3.3 Sondersituation: Wenn die Konsole vollgeschrieben ist.	5
3.4 Beispiel: Korrektur	6
4 Quellcode (C#)	7

1 Lösungsidee

Für die Lösungsidee habe ich mir vorgenommen, einen eigenen Texteditor zu schreiben, um Zara beim Erstellen von Hopsitexten zu unterstützen. Um die Arbeit zu vereinfachen, soll der Editor folgende Funktionen unterstützen:

- Rechtzeitige Warnung bei ungültiger Eingabe
 - Das Programm soll den Endnutzer rechtzeitig visuell warnen, falls ein Buchstabe eingegeben wird, der zu einer anderen Sprunglandung führt.
- Voraussetzende Landemarkierung
 - Das Programm soll die nächste Stelle anzeigen, an der ein Sprung landen wird.
 - Diese Landemarkierung muss aktualisiert werden, wenn ein Zeichen ohne Sprungwert eingefügt wird (z. B. „“, „“, „“, Leerzeichen).
- Vorhersage verbotener Zeichen
 - Dadurch soll man frühzeitig informiert werden, welche Zeichen am Sprunglandeort nicht erlaubt sind.

Da Zara mit diesem Programm besonders gut arbeiten soll, wird vor allem die Benutzeroberfläche (durch Farben und Positionsanzeigen) priorisiert, um eine gute Kommunikation zwischen Programm und Endnutzer zu ermöglichen. Außerdem soll die Möglichkeit bestehen, bereits geschriebene Texte einfach einzufügen und so nach Fehlern suchen zu können.

Um die Sprungweite von Zeichen zu bestimmen, wird die Tabelle aus der JWINF verwendet.

Es soll auch möglich sein, Zeichen zu entfernen, um Fehler zu korrigieren. Da lange Texte erstellt werden sollen, muss dieses Programm die Möglichkeit bieten, weiter schreiben zu können, auch wenn die Konsole bereits vollgeschrieben ist. Dafür habe ich mir überlegt, dass man mit der Zeichensetzung von oben wieder anfängt, wobei die älteren Zeilen nicht mehr sichtbar, aber weiterhin gespeichert sind.

2 Umsetzung

Für die Umsetzung wird C# in Kombination mit .NET 8 verwendet. C# eignet sich am besten für diese Aufgabe, da die Sprache im System-Namespace viele voreingebaute Konsolenfunktionen unterstützt, wodurch man die Konsole als Benutzeroberfläche verwenden werden kann (ähnlich wie der Texteditor VIM). Dadurch vermeidet man eine aufwändige grafische Benutzeroberfläche zu erstellen.

Von der Klasse `System.Console` werden folgende Eigenschaften verwendet:

`Console.ReadKey`: Funktion, liest die Zeichen von der nächsten eingegebenen Taste ab. Das eingegebene Zeichen kann zum Datentyp `char` umgewandelt werden.

`Console.SetCursorPosition`: Diese Funktion erlaubt das Platzieren des Zeigers an einer bestimmten Position um Zeichen an der richtigen Stelle anzuzeigen.

`Console.WindowHeight`: Attribut, gibt die Anzahl der Zeichen, die vertikal reinpassen, zurück. Es wird verwendet, um zu berechnen, wann der Platz nicht mehr ausreicht.

`Console.WindowWidth`: Attribut, gibt die Anzahl der Zeichen, die horizontal reinpassen, zurück. Es wird verwendet, um den Moment zu berechnen, an dem Zeichen in die nächste Zeile wechseln müssen.

`Console.BackgroundColor`: Attribut mit Setter, das verwendet wird, um die Hintergrundfarben für die Sprungpositionen und den verbotenen Buchstaben hervorzuheben.

Durch die Kombination dieser Funktionen kann eine einfache Benutzeroberfläche erstellt werden, die alle vorgesehenen Ziele erfüllt.

2.1 Funktionen, um die Buchstaben zu konvertieren

Anhand der Tabelle aus der Junior Aufgabe 2 wird eine Funktion namens „`GetValue`“ erstellt, die jedem einzelnen Buchstaben einen eindeutigen Wert zuweist.

Für die Eingabe wird ein Wert vom Datentyp `Byte` (`char -> byte`) benötigt. In der Funktion eignet sich eine `switch/case`-Anweisung am besten, um viele `if`-Abschnitte zu vermeiden. Da `char` als UTF-16 gespeichert wird, liegen die großen Buchstaben alphabetisch zwischen den Werten 65 und 90. Die Kleinbuchstaben befinden sich zwischen 97 und 122. Daher kann man in beiden Fällen überprüfen, ob die Zahl in diesem Bereich liegt. Falls es so ist, muss man den UTF-16 Wert durch die Summe aus dem Minimum Wert (Großbuchstabe: 65, Kleinbuchstabe: 97) und 1 abziehen, um den Sprungwert zu erhalten. Für die Zeichen Ä, Ö, Ü muss man es einzeln definieren und so den Wert zurückgeben. Außerdem habe ich mir vorgenommen, dass die Funktion für Zeichen ohne Sprungwert: 0, Rücktaste: -1, Enter: -2 und Tab: -3 zurückgibt, um solche Werte separat zu verarbeiten.

Es wird außerdem eine weitere Funktion benötigt, die den Wert rückgängig macht („`ReverseValue`“). Als Argument wird die Differenz von zwei Landepositionen als Integer benötigt. Die Funktion gibt anhand der `switch/case`-Anweisung den UTF-16 Wert der Buchstaben zurück. Dieser Buchstabe wird der verbotene Buchstabe sein, welchen man bei der nächsten Sprungposition nicht verwenden sollte.

2.2 Anzeigepositionen

Alle Positionen (auch der Eingabezeiger: `pos`) bekommen einen Integer Wert, der ihre Position bestimmt. Mithilfe der selbst erstellten Funktion `GetLocation` kann dieser Integer Wert in `x` und `y` Werte umgewandelt werden. `GetLocation` bestimmt, wo der Zeiger sein soll, damit der Text an der richtigen Stelle angezeigt wird. Wenn die Konsole vollgeschrieben ist, beginnt `GetLocation` erneut von oben. Diese Funktion wird die erste und letzte Zeile in der Konsole nicht beschriften, da die erste Zeile für Nachrichten reserviert ist und die letzte verwendet werden soll, um, sobald man bei der vorletzten Zeile ist, die verbotenen Zeichen bei der letzten Zeile anzuzeigen. Durch die Kombination von `GetLocation` und `Console.SetCursorPosition` können alle Zeichen richtig platziert werden.

Die beiden Sprungpositionen habe ich in „nächste Sprungposition“ (`nextPos`) und „nächste Referenz-Sprungposition“ (`targetPos`) aufgeteilt. Beide erhalten unterschiedliche Farben (`nextPColor`: rot; `targetPColor`: blau) und jeweils einen Integer Wert, der ihre Position ausmacht. Je nachdem wo die Positionen landen, wechseln sich die Rollen. Das heißt, wenn `nextPos` größer als `targetPos` ist (also

wenn targetPos vor nextPos liegt), werden die Werte vertauscht und targetPos wird zur neuen nextPos. Dadurch soll erreicht werden, dass nextPos immer vor (bzw. kleiner als) targetPos ist. Die Farben Rot und Blau werden bei dem Rollenwechsel auch getauscht. Bsp.:

- nextPos = 4; nextPColor = Rot; targetPos = 6; targetPColor = Blau;
- Nutzer gibt bei Position 4 "e" ein -> e hat Sprungwert von 5 -> nextPos += 5
- nextPos !< targetPos -> 9 ist nicht kleiner als 6 -> Rollenwechsel
- nextPos = 6; nextPColor = Blau; targetPos = 9; targetPColor = Rot;

Außerdem habe ich die Funktion Warning erstellt, die die andere Sprungposition (nextPos) markiert (mittels GetLocation) und die verbotenen Zeichen anzeigt. Diese verbotenen Zeichen werden durch die Differenz zwischen der nächsten Sprungposition und der anderen Sprungposition berechnet und mithilfe der Funktion ReverseValue in UTF-16 Zeichen umgewandelt.

2.3 Weitere Funktionen

Die Funktion ShiftValues verschiebt alle Sprungpositionen um einen bestimmten Wert (mit GetLocation) verschieben. Dieser Wert ist davon abhängig, ob man diese Funktion durch die Rücktaste auslöst (-1, Markierungen werden nach links verschoben), oder durch ein Zeichen ohne Sprungwert (+1, Markierungen werden nach rechts verschoben). Die Werte werden alle verschoben, da ein Zeichen ohne Sprungposition beim Hopsen ignoriert wird und deshalb die voraussichtliche Sprungposition neu berechnet werden muss, indem man sie verschiebt. Diese Funktion hat einen Einfluss auf die Darstellung und auf die Variablen.

Anschließend gibt es noch die Pause Funktion, um Hinweise anzuzeigen (vom Typ string text). Die Funktion hält das Programm an und wartet, bis die Tabulatortaste gedrückt wird, um fortzufahren.

Die Funktion deleteLine entfernt eine bestimmte Zeile (ebenfalls mithilfe von GetLocation), falls der Platz nicht mehr ausreicht. Der Inhalt bleibt weiterhin gespeichert, wird jedoch nicht mehr angezeigt. Dies sollte sicherstellen, dass der Benutzer das Programm auch bei vollem Konsolenfenster weiterhin verwenden kann. Die gelöschten Inhalte sollen in der Konsole nicht wiederherstellbar sein, da man davon ausgehen kann, dass man nicht mehr zurück geht. Eine Korrektur bei einem Zeichenfehler, der vor vielen hundert Zeichen passiert ist, könnte eine große Auswirkung haben. Deshalb soll auf Möglichkeiten zur Korrektur von älteren Fehlern verzichtet werden.

2.4 Stapelspeicher

Damit die Rücktaste auch alle vorherigen Aktionen rückgängig machen kann, werden folgende Werte im Stapelspeicher gespeichert.:

targetMoments: Alle Momente, in denen die Rollen beider Sprungpositionen getauscht werden (da nextPos < targetPos sein sollte), werden im Stapelspeicher targetMoments gespeichert. Dies ermöglicht es, die vorherigen Sprungpositionen nachzuvollziehen.

pointerMoments: Ebenso werden alle Momente gespeichert in denen die nächste Sprungposition erreicht wurde, jedoch trotzdem noch vor der anderen Sprungposition liegt (also ohne Rollenwechsel). Diese Momente werden im Stapelspeicher pointerMoments gespeichert, um auch hier die vorherigen Sprungpositionen rekonstruieren zu können.

SkipMoments: Schließlich werden auch alle Momente gespeichert, in denen ein Zeichen ohne Sprungwert eingegeben wurde, um die daraus resultierenden Verschiebungen rückgängig machen zu können.

2.5 Eingabe & Schleife

Am Anfang der Funktion werden zunächst beide Landepositionen markiert. Anschließend beginnt eine Endlosschleife ohne Abbruchmöglichkeit (da dieses Programm dauerhaft laufen soll). Zuerst wird überprüft, ob noch genug Platz vorhanden ist (falls nicht, wird deleteLine ausgelöst, um eine Zeile von oben zu entfernen und Platz zu schaffen). Danach wird die Position zurückgesetzt und das Programm wartet auf eine Eingabe (Variable: key).

Nach der Eingabe wird der Eingabewert (Variable: keyValue) über die Funktion GetValue! ermittelt. Anhand der Eingabe wird dann entschieden, was gemacht wird.

Tabulator (-3): Der Text wird gespeichert (Der Benutzer/Die Benutzerin wird mithilfe der Funktion Pause darauf hingewiesen).

Enter (-2): Da passiert nichts, da sonst die Positionen durch die neue Zeile durcheinander gebracht werden.

Rücktaste (-1): Dadurch wird der zuletzt eingegebene Zeichen gelöscht. Anschließend werden in alle Stapelspeicher geschaut, ob es eine besondere Aktion gab, um diese ebenfalls rückgängig zu machen.

Zeichen ohne Sprungwert (0): Bei einem Zeichen ohne Sprungwert werden die nächsten Sprungmarkierungen und verbotenen Zeichen nach rechts verschoben. (Mithilfe von ShiftValue)

Sonst wird noch geschaut, ob die aktuelle Position die nächste Sprungposition ist. Ist es der Fall, wird der Benutzer über die Funktion Pause! gewarnt, wenn dieser Eingabewert dazu führt, dass beide Springer in der gleichen Position landen. Ist es nicht der Fall, so werden die neuen Sprungpositionen berechnet und geschaut, ob es einen Rollenwechsel gibt (Erinnerung: $\text{nextPos} < \text{targetPos}$).

Alle Eingaben (außer Enter und Eingaben mit verbotenen Zeichen) werden notiert (unter Variable "text") und beim Speichern auf der Festplatte geschrieben.

3 Beispiele

3.1 Beispiele aus der PDF-Datei

Als Beispiele für einen funktionierenden Hopsitext habe ich die beiden Beispiele aus der Angabe entnommen. Dabei kann man erkennen, dass die Markierungen identisch zu den in der Aufgabenstellung gezeigten Text sind.

Zusätzlich kann man im Bild die Vorhersage sehen (Bild 1: Unter der blauen freien Fläche, mit dem Buchstaben I). Im Beispielttext aus der jwinf (Bild 2) kann man erkennen, dass der nächste verbotene Buchstabe "K" ist und der Zeiger schon bereits in dieser Position ist. Dadurch weiß man, dass man "K"

nicht eingeben darf, und man eine Fortsetzung, die nicht mit "K" beginnt, auswählen muss. (Wie z.B. "Die Gewinnerin wird...")

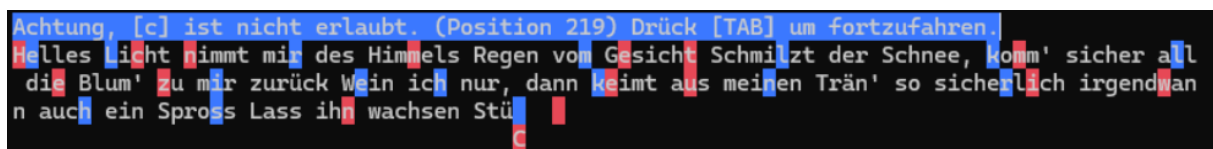


3.2 Beispiel mit ungültigen Zeichen

Als weiteres Beispiel habe ich mir einen Text ausgesucht, wo beide Springer in der gleichen Endposition landen werden. Dazu habe ich die Lyrics eines Covers vom Samina König ausgewählt (es funktionieren natürlich auch andere Werke). Im ersten Bild sieht man die erste Strophe des Werkes und im zweiten Bild die Warnung in der Konsole (erste Zeile).

Lyrics

Helles Licht nimmt mir des Himmels Regen vom Gesicht
 Schmilzt der Schnee, komm' sicher all die Blum' zu mir zurück
 Wein ich nur, dann keimt aus meinen Trän' so sicherlich
 irgendwann auch ein Spross
 Lass ihn wachsen Stück für Stück

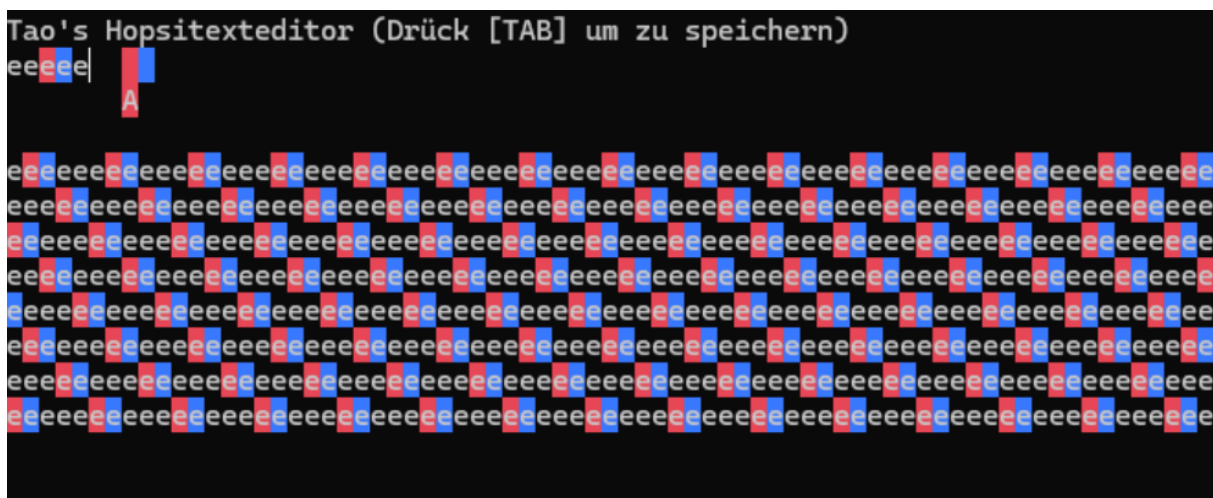
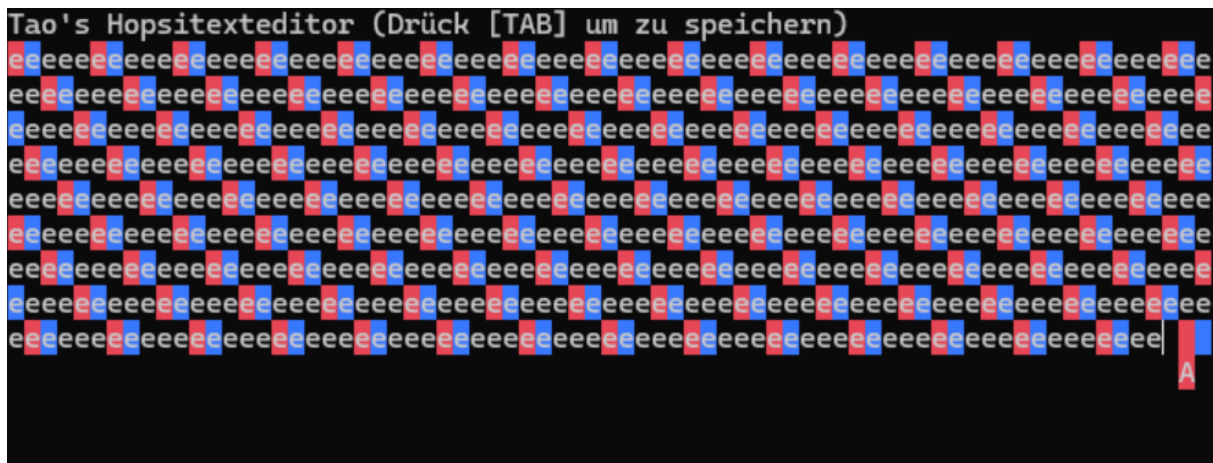


(vgl.: Samina König: "Haru yo, Koi", lyrics.lyricfind.com/lyrics/selphius-haru-yo-koi 09.11.2024)

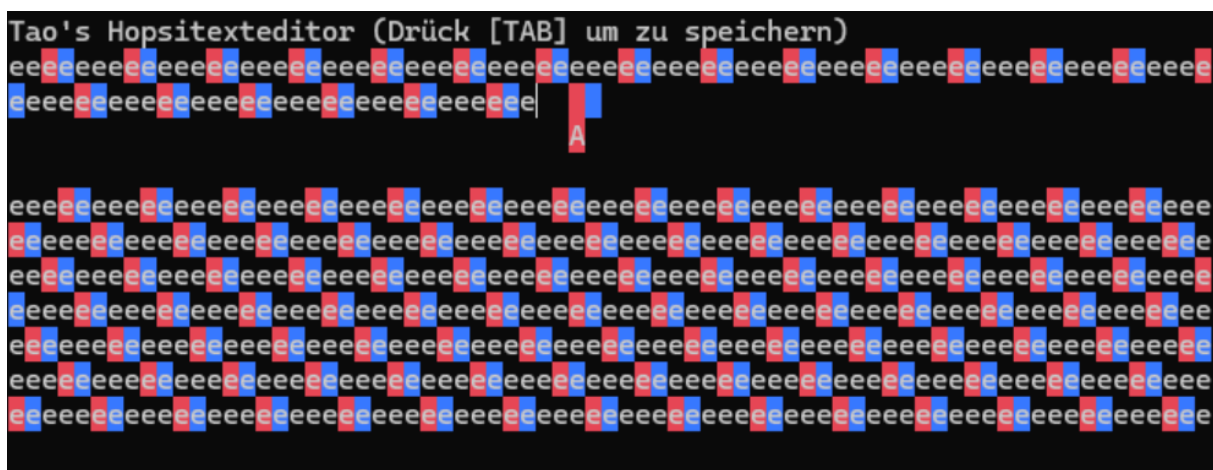
Das soll die Benutzer*innen daran hindern, weiterzuschreiben, bis man diese Warnung wahrgenommen hat. Dadurch werden Folgefehler verhindert, die ohne Unterbrechung passieren könnten (vor allem für Personen, die schnell an der Tastatur tippen).

3.3 Sondersituation: Wenn die Konsole vollgeschrieben ist.

Ein anderer Sondermoment ist, wenn die Konsole vollgeschrieben ist. In diesem Fall werden beim nächsten Zeilenwechsel die ersten Zeilen vollständig entfernt, damit die Endnutzerin (Zara) weiterschreiben kann.



Schreibt der Benutzer weiter, werden auch die danach folgenden Zeilen ebenfalls entfernt, um so das Schreiben zu ermöglichen. Der Inhalt der entfernten Abschnitte bleibt weiterhin gespeichert und wird auch beim Speichern in der Datei angezeigt.



3.4 Beispiel: Korrektur

Zum Schluss habe ich einen Fall ausgesucht, bei dem ein Tippfehler korrigiert wird. Dafür habe ich den Text “Hallo Welt!” ausgesucht, bei dem jedoch aus Versehen statt einem “o” ein “p” geschrieben wurde.

Da "!" keinen eigenen Wert hat und beim Hopsen übersprungen wird, verschiebt sich bei der Entfernung des Zeichens die Markierung um eine Position nach links.

Anschließend kommen wir zu einer der Sprungposition. Wenn dieses Zeichen entfernt wird, muss auch die darauffolgende Markierung ebenfalls entfernt werden und die Vorhersage neu berechnet werden.

Die rote Markierung wurde entfernt und die Vorhersage wurde auf "F" berechnet. Da die anderen Buchstaben ("Wel") alle einen Sprungwert haben, können die entfernt werden, ohne dass eine Verschiebung durchgeführt werden muss.

Nun kommt man zu einem Zeichen, auf dem man nicht landen kann (Leerzeichen). Deshalb verschiebt man beim Entfernen von Leerzeichen wieder alle Markierungen um eine Position nach links.

Sobald man beim Buchstaben angekommen ist, kann man ihn entfernen und so den Text korrigieren.

Nach der Korrektur kann man diesen Text wie gewohnt weiterschreiben.

4 Quellcode (C#)

```
Console.WriteLine("Tao's Hopsitexteditor (Drück [TAB] um zu speichern)");

Stack<int> pointerMoments = new Stack<int>();
Stack<int> targetMoments = new Stack<int>();
Stack<int> skipMoments = new Stack<int>();
string text = "";
```

```
int pos = 0;
ConsoleKeyInfo key;
int keyValue;

int nextPos = 0;
ConsoleColor nextPColor = ConsoleColor.Red;

int targetPos = 1;
ConsoleColor targetPColor = ConsoleColor.Blue;

(int, int) GetLocation(int Position)
{
    // lineare Position zur Koordinate umwandeln
    int LetterPos = Position % ((Console.WindowHeight - 2) * Console.WindowWidth);
    int y = LetterPos / Console.WindowWidth + 1;
    int x = LetterPos % Console.WindowWidth;
    return (x, y);
}

int GetValue(byte B)
{
    // Eingabe mithilfe der Tabelle von Junioraufgabe 2 in Werte umwandeln
    switch (B)
    {
        // ABC Großbuchstabe
        case >= 65 and <= 90:
            return (int)B - 64;
        // abc Kleinbuchstabe
        case >= 97 and <= 122:
            return (int)B - 96;
        // ä
        case 228:
        case 196:
            return 27;
        // ö
        case 246:
        case 214:
            return 28;
        // ü
        case 252:
        case 220:
            return 29;
        // ß
        case 223:
            return 30;
        // Backspace
        case 8:
            return -1;
        // Enter
        case 13:
            return -2;
        // Tab
        case 9:
            return -3;
        // ignorieren
        default:
            return 0;
    }
}

byte ReverseValue(int value)
{
    /* Werte mithilfe der Tabelle von Junioaufgabe 2 in bytes umwandeln.
```



```

* Die zurückgegebene Bytes stehen für den bestimmten char (Nach UTF-8)
*/
switch (value)
{
    // A - Z
    case < 27:
        return (byte)(value + 64);
    // Ä
    case 27:
        return 228;
    // Ö
    case 28:
        return 246;
    // Ü
    case 29:
        return 252;
    // ß
    case 30:
        return 223;
    // failsafe
    default:
        Console.WriteLine("ReverseValue received " + value);
        return 0;
}
}
void warning()
{
    // platziert einen Hinweis unter der nächsten SprungPosition und markiert die nächste
    TargetPoint
    int x, y;
    Console.BackgroundColor = targetPColor;
    (x, y) = GetLocation(targetPos);
    Console.SetCursorPosition(x, y);
    Console.Write(" ");

    (x, y) = GetLocation(nextPos);
    Console.SetCursorPosition(x, y + 1);
    Console.BackgroundColor = ConsoleColor.Red;
    Console.Write((char)ReverseValue(targetPos - nextPos));
    (x, y) = GetLocation(pos);
    Console.SetCursorPosition(x, y);
}
void ShiftValues(ConsoleKeyInfo key, int amount)
{
    // Bewegt alle Werte um Anzahl "amount"
    // mit dem Ziel, dass bei Zeichen die ignoriert werden weiterhin die Stellen richtig
    // angezeigt werden.

    // nächste SprungPosition entfernen
    char text = ' ';
    if ((pos - amount) == nextPos) text = key.KeyChar;
    int x, y;
    (x, y) = GetLocation(nextPos);
    Console.SetCursorPosition(x, y);
    Console.BackgroundColor = ConsoleColor.Black;
    Console.Write(text);

    // Hinweis entfernen
    Console.SetCursorPosition(x, y + 1);
    Console.Write(" ");

    // targetPoint entfernen

```

```

(x, y) = GetLocation(targetPos);
Console.SetCursorPosition(x, y);
Console.Write(" ");

// neue SprungPosition markieren
(x, y) = GetLocation(nextPos + amount);
Console.SetCursorPosition(x, y);
Console.BackgroundColor = nextPColor;
Console.Write(" ");
Console.BackgroundColor = ConsoleColor.Black;

// SprungPosition und targetPoint bewegen.
nextPos += amount;
targetPos += amount;

// TargetPoint und Hinweis neu markieren und anzeigen.
warning();
}
void Pause(string text)
{
    // Pausiert das Programm bei wichtigen Hinweisen.
    Console.SetCursorPosition(0, 0);
    Console.Write(text + " Drück [TAB] um fortzufahren.");
    ConsoleKeyInfo input;
    do
    {
        input = Console.ReadKey(true);
    } while (input.KeyChar != 9);
    // Text entfernen
    Console.BackgroundColor = ConsoleColor.Black;
    Console.SetCursorPosition(0, 0);
    Console.Write(new string(' ', 31 + text.Length));
    return;
}
void deleteLine()
{
    // Bei fehlender Platz Zeile komplett löschen.
    // Die eingegebenen Zeichen sind weiterhin gespeichert,
    // werden jedoch nicht mehr angezeigt.
    int x, y;
    (x, y) = GetLocation(pos + 2 * Console.WindowWidth);
    Console.SetCursorPosition(0, y);
    Console.BackgroundColor = ConsoleColor.Black;
    Console.Write(new string(' ', Console.WindowWidth));
}

// flexible Koordinaten
int x, y;

// Erste Sprungposition markieren
(x, y) = GetLocation(nextPos);
Console.SetCursorPosition(x, y);
Console.BackgroundColor = nextPColor;
Console.Write(" ");

// Zweite Sprungposition markieren
warning();

while (true)
{
    // Auslösen wenn die zweite Seite erreicht wurde und die Konsole Platz braucht.
    if (pos / (Console.WindowWidth * (Console.WindowHeight - 5)) >= 1 && pos %

```

```

Console.WindowWidth == 0)
{
    deleteLine();
}
// Zeiger auf die richtige Stelle zurücksetzen.
(x, y) = GetLocation(pos);
Console.SetCursorPosition(x, y);
if (pos == nextPos) Console.BackgroundColor = nextPColor;
else Console.BackgroundColor = ConsoleColor.Black;
key = Console.ReadKey();
keyValue = GetValue((byte)key.KeyChar);
if (keyValue == -3)
// Bei TAB -> Speichern
{
    File.WriteAllText("TaoZHopsitextoutput.txt", text);
    Pause("Datei wurde als TaoZHopsitextoutput.txt gespeichert.");
    Console.SetCursorPosition(x, y);
    continue;
}
else if (keyValue == -2)
// Enter -> ignorieren.
{
    continue;
}
else if (keyValue == -1)
// Backspace -> Zeichen entfernen
{
    if (pos == 0) continue;
    text = text.Remove(text.Length - 1);
    if (pos % Console.WindowWidth == 0)
    {
        (x, y) = GetLocation(pos - 1);
        Console.SetCursorPosition(x, y);
    }
    if (pos == nextPos)
    {
        Console.BackgroundColor = ConsoleColor.Black;
    }
    Console.Write(" ");
    pos -= 1;
    // Überprüfen, ob die letzten Zeichen vorher einer der SprungPosition waren
    if (targetMoments.TryPeek(out int targetMoment) && pos == targetMoment)
    {
        (x, y) = GetLocation(targetPos);
        Console.SetCursorPosition(x, y);
        Console.Write(" ");
        (x, y) = GetLocation(nextPos);
        Console.SetCursorPosition(x, y + 1);
        Console.Write(" ");
        ConsoleColor tempColor = targetPColor;
        targetPos = nextPos;
        targetPColor = nextPColor;
        nextPColor = tempColor;
        nextPos = targetMoments.Pop();
        warning();

        (x, y) = GetLocation(pos);
        Console.SetCursorPosition(x, y);
        Console.BackgroundColor = nextPColor;
        Console.Write(" ");
    }
    // Überprüfen, ob es einer der SprungPositionen war

```

```

else if (pointerMoments.TryPeek(out int pointerMoment) && pos == pointerMoment)
{
    (x, y) = GetLocation(nextPos);
    Console.SetCursorPosition(x, y);
    Console.Write(" ");
    Console.SetCursorPosition(x, y + 1);
    Console.Write(" ");
    nextPos = pointerMoments.Pop();

    (x, y) = GetLocation(pos);
    Console.BackgroundColor = nextPColor;
    Console.SetCursorPosition(x, y);
    Console.Write(" ");
    warning();
}
// Überprüfen, ob der vorherige Zeichen einer der ignorierten Zeichen war.
else if (skipMoments.TryPeek(out int skipMoment) && skipMoment == (pos + 1))
{
    skipMoments.Pop();
    ShiftValues(key, -1);
}
continue;
}
else if (keyValue == 0)
// ignorierbare Zeichen -> alle Zeichen müssen um eine Position verschoben werden.
{
    text += key.KeyChar;
    pos += 1;
    skipMoments.Push(pos);
    ShiftValues(key, 1);
    continue;
}
else if (pos == nextPos)
// Spezialfall: der nächste Zeichen ist einer der SprungPosition
{
    // Überprüfen, ob der Wert zu einem Hopsitext mit gleicher Ausgang führt.
    if (keyValue == (targetPos - nextPos))
    {
        // Zeichen entfernen und den Nutzer darauf hinweisen.
        Console.SetCursorPosition(x, y);
        Console.Write(" ");
        Pause("Achtung, [" + key.KeyChar + "] ist nicht erlaubt. (Position " + (pos + 1) +
"");
        continue;
    }
    text += key.KeyChar;
    Console.BackgroundColor = ConsoleColor.Black;
    Console.SetCursorPosition(x, y + 1);
    Console.Write(" ");
    if ((nextPos + keyValue) > targetPos)
    {
        // Fall: der vorherige Sprungposition kommt nach der zweiten Sprungposition ->
        Rollenwechsel
        targetMoments.Push(pos);
        int tempPos = nextPos;
        ConsoleColor tempColor = nextPColor;
        nextPos = targetPos;
        nextPColor = targetPColor;
        targetPos = (tempPos + keyValue);
        targetPColor = tempColor;
    }
}
else

```

```
    {
        // Fall: der vorherige Sprungposition bleibt vor der zweiten Sprungposition ->
        kein Rollenwechsel
        pointerMoments.Push(pos);
        nextPos += keyValue;
        (x, y) = GetLocation(nextPos);
        Console.SetCursorPosition(x, y);
        Console.BackgroundColor = nextPColor;
        Console.Write(" ");
    }
    pos += 1;
    warning();
    continue;
}
else
    // Sonst Zeichen setzen.
    {
        text += key.KeyChar;
    }
    pos += 1;
}
```